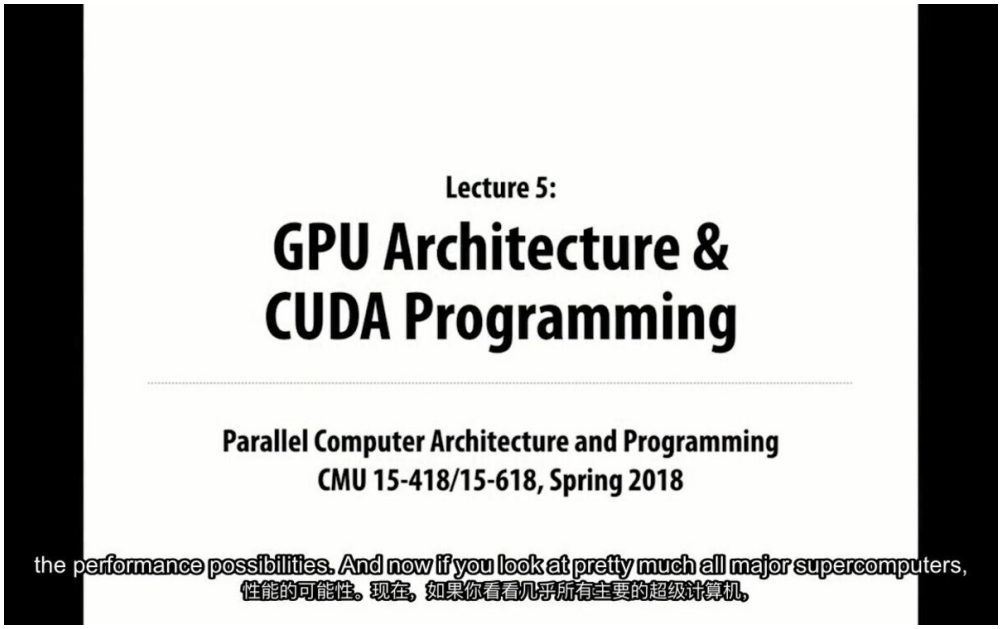
课程笔记

GPU 架构与 CUDA 编程：把算力装进线程块

CMU 15-418/618 并行计算机架构与编程 · Lecture 7（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 8 秒
视频链接：`https://www.bilibili.com/video/BV1Hr4y1Z7tn/`

目录

1 GPU：从图形卡到超级计算机的心脏 3

1.1 基本结构 . . . 3
1.2 历史：为三维游戏而生 . . . 3
1.3 本章小结 . . . 5

2 图形管线：天然的数据并行 5

2.1 实体与图元 . . . 5
2.2 从三维到二维的管线 . . . 6
2.3 片元处理与材质复杂度 . . . 7
2.4 着色器：可编程的管线阶段 . . . 8
2.5 本章小结 . . . 8

3 从图形卡到通用计算 9

3.1 历史背景 . . . 9
3.2 把科学计算伪装成图形操作 . . . 9
3.3 为什么不能让操作系统调度 . . . 11
3.4 CUDA 的定位 . . . 12
3.5 代价：功耗与价格 . . . 13
3.6 本章小结 . . . 13

4 CUDA 的抽象：线程、块与网格 13

4.1 先澄清一个术语陷阱 . . . 13
4.2 三维的线程索引 . . . 14
4.3 block 与 grid . . . 14
4.4 核函数 . . . 15
4.5 三个修饰符 . . . 16
4.6 block 尺寸的硬限制 . . . 17
4.7 越界处理 . . . 17
4.8 本章小结 . . . 17

5 内存层次与共享内存 18

5.1 地址空间是分离的 . . . 18
5.2 三类内存 . . . 19
5.3 本章小结 . . . 20

6 卷积示例：从 3n 次读取到一次 20

6.1 版本一：一个线程一个输出 . . . 20
6.2 版本二：先用共享内存装一遍 . . . 21
6.3 为什么必须同步 . . . 22
6.4 本章小结 . . . 22

7 抽象回顾与执行细节 **23**
7.1 CUDA 抽象的两层 23
7.2 warp 与 SMM 23
7.3 本章小结 24

8 总结与延伸 **25**

1 GPU：从图形卡到超级计算机的心脏

讲者开场给出一个判断：GPU 是过去几十年高性能计算领域最令人兴奋的进展之一，因为它把性能可能性整体抬高了。今天几乎所有主要超级计算机的主体，都是一堆 GPU 拼起来。

这份算力并非自动可得。讲者点明本讲的双重主题：在某一层抽象上，你甚至不需要理解硬件就能用起来；但要拿到真正的性能，你必须知道下面发生了什么。

1.1 基本结构

回顾前几讲的结构：一颗很大的芯片上有许多处理器。每个处理器在很粗略的层面像一个常规 CPU——能通过 SIMD 指令支持多个 ALU，也能像 Intel 所谓超线程（更恰当的叫法是同时多线程）那样支持多条线程。

NVIDIA 把这些单元称作 SM（streaming multiprocessor，流式多处理器）。讲者展示的图上有 16 个，那是稍早的型号；课程用的 GHC 机器上有 20 个。

它们连接的内存，在内存世界里相对很小，但带宽相当可观。

整卡的原始 ALU 总量可以这样估：

$$N_{\text{ALU}} = N_{\text{SM}} \times N_{\text{ALU per SM}}$$

- N_{SM}：芯片上 SM 的数量，课程机器为 20；
- $N_{\text{ALU per SM}}$：每个 SM 的 ALU 数，课程 GPU 为 32；
- N_{ALU}：整卡可同时执行的算术操作数上限。

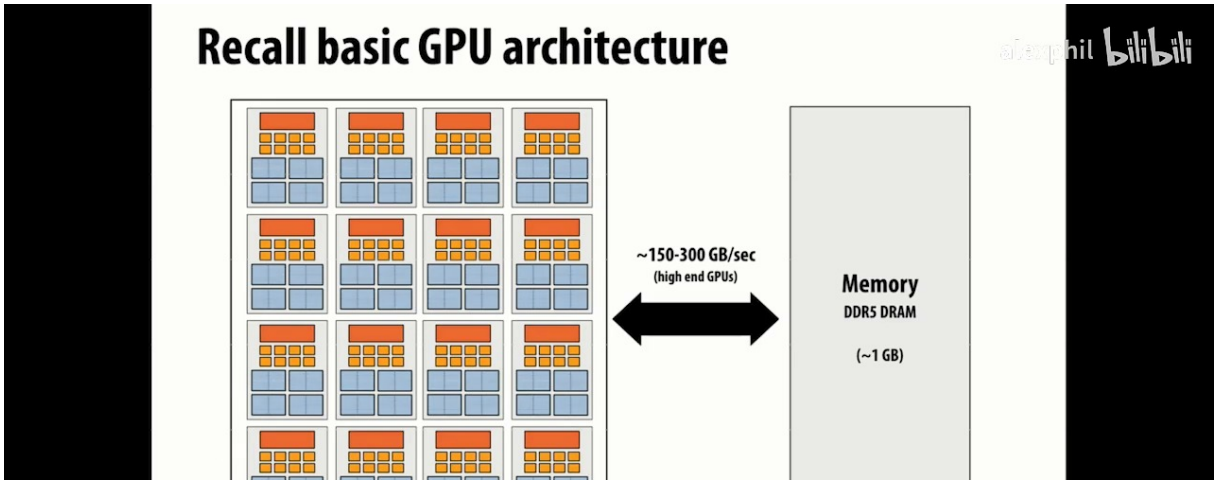


图 1: 基本 GPU 架构：多个 SM 共享高带宽内存[1]

1.2 历史：为三维游戏而生

讲者从历史讲起。GPU 最初的设计目的是加速三维游戏这类应用。

[1] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

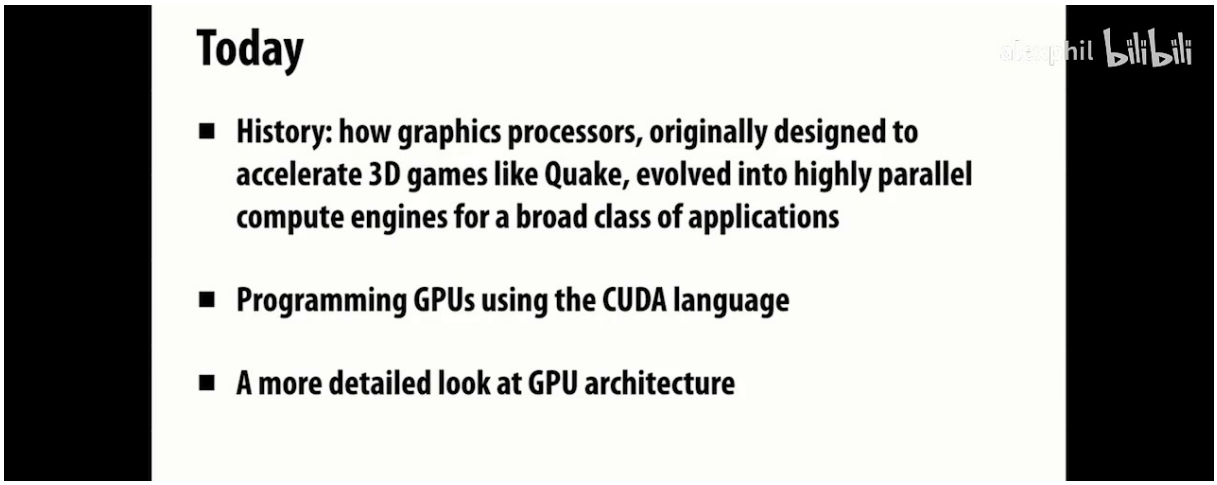


图 2: GPU 历史：图形处理器原本被设计来加速三维应用[2]

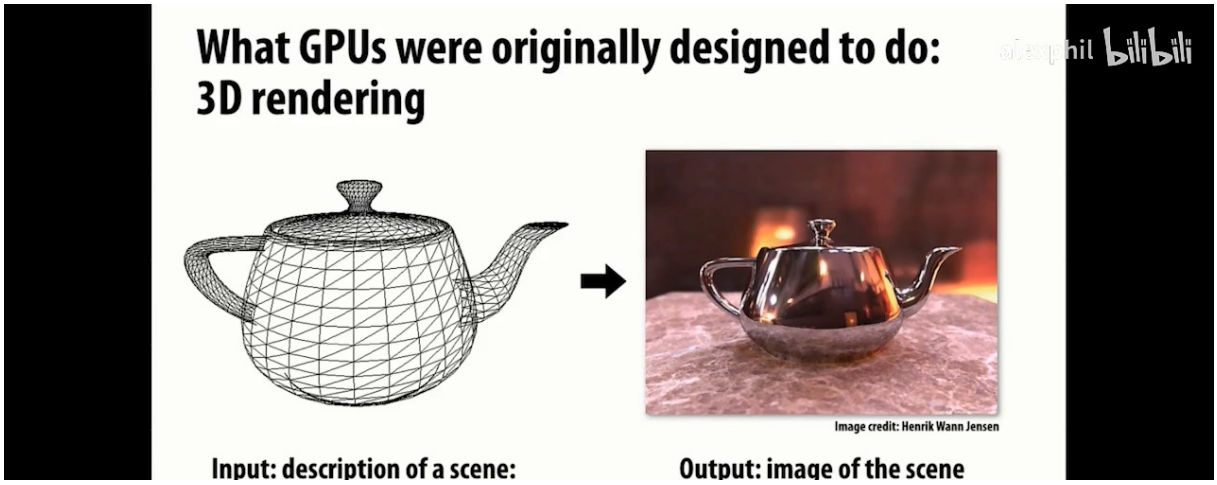


图 3: GPU 最初的用途：三维渲染[3]

实时渲染的帧率预算决定了一切

每秒 30 帧意味着每帧只有约 33 毫秒，其中还要扣掉物理模拟、输入处理与合成。这个固定预算解释了 GPU 为何偏爱近似手段（如纹理映射）而不做精确的物理建模：在预算内给出足够真实的观感，比算出精确的光照更重要。

至今这仍是它的主要用途：在高端的 GPU 上实时（约每秒 30 帧）渲染画面。

[2]视频画面时间区间：00:01:30–00:01:45。若为裁剪图，时间区间仍对应原视频画面。
[3]视频画面时间区间：00:03:30–00:03:45。若为裁剪图，时间区间仍对应原视频画面。



图 4: GPU 至今仍在做的事：高端 GPU 上的实时渲染[4]

追求真实感是一条指数级变难的路。讲者用一张风景图逐项举例：远处的景物比近处的略微模糊，这是大气效果；山间那些丝缕状的云，需要先跑一个扩散模型再做渲染。光照、光源位置、反射，每增加一层真实感就增加一层计算。若再加一个在草丛中跑动的男孩，还要渲染符合人体运动规律的姿态；若起风，还得让每根草摆动。

这些需求不只来自游戏，也来自建筑可视化——客户想绕着尚未建成的楼房从各个角度观察，还想看到云的倒影落在那面玻璃幕墙上。每一步都在增加复杂度。

1.3 本章小结

GPU 的算力来自大量 SM 的堆积与高带宽内存。它原本为实时三维渲染而生，真实感需求的指数级增长持续把它推向更高算力。理解这一点才能明白后面 CUDA 为何采用那样的抽象——它的形状是从图形管线继承来的。

2 图形管线：天然的数据并行

2.1 实体与图元

图形学要处理的对象是表面。讲者说，实时图形用三角网格表示表面：给定每个顶点在 x、y、z 上的三个坐标，每三个顶点组成一个三角形，一系列三角形构成物体的外观。

[4] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

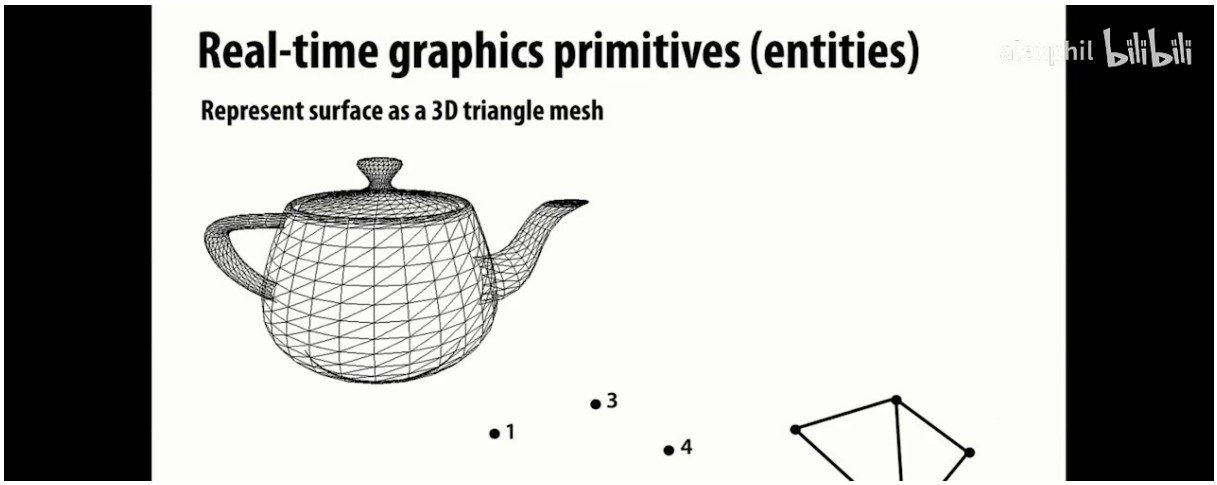


图 5: 实时图形的实体：用三维三角网格表示表面[5]

2.2 从三维到二维的管线

渲染一张图的过程可以拆成几步。

输入是顶点列表，它们存在于三维空间中。接下来要把这些三角形投影到二维屏幕上，如同从一个观察者或摄像机的视角看过去。这个映射取决于观察者的角度与位置。

投影之后得到一系列图元，在此例中是被压平到画面上的三角形。再往下，因为要转成离散像素，每个三角形内部要划分成小方格，并为每一格赋予颜色值。最后完成着色。

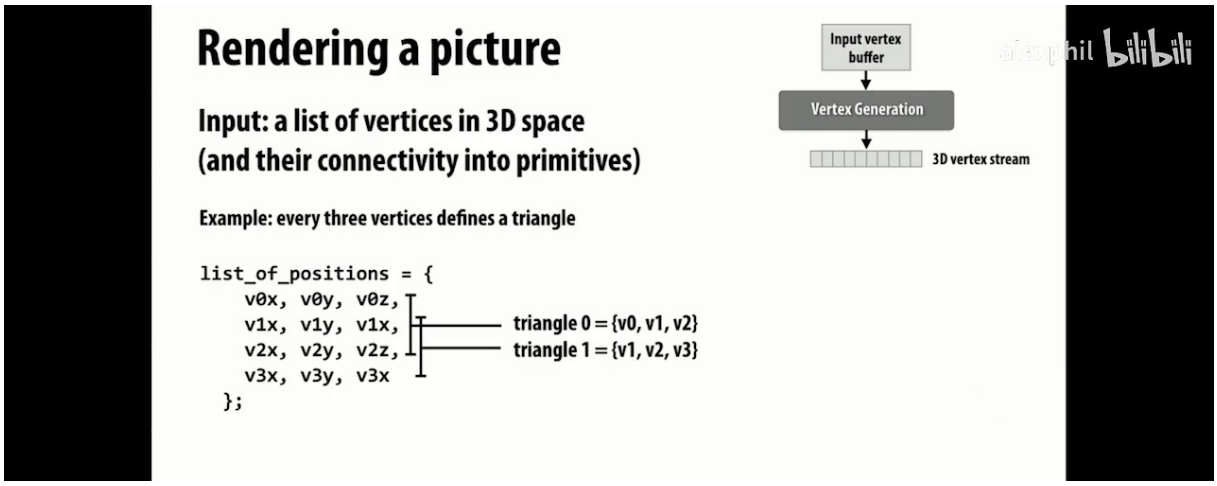


图 6: 渲染一张图：从顶点缓冲经投影到像素的完整管线[6]

[5] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

图形管线的每一步都是天然的数据并行

处理顶点时，每个顶点与其他顶点无关；处理三角形时，遮挡计算只依赖各自的深度关系；处理像素时，每个小方格独立着色。对象在变（顶点、三角形、像素），操作也在变（投影、遮挡、着色），但”一堆东西各自独立地被处理”这个结构始终不变。GPU 的硬件形态正是为这个结构设计的。

2.3 片元处理与材质复杂度

讲者接着讨论片元处理，也就是逐像素计算光照。他用一系列球体展示材质的差异：大理石的光不只从表面反射，还会进入表面内部再折回，同时具有半透明性，薄的地方会透光；金属则有镜像效果，理论上要反向追踪被反射的一切；皮肤更难，皮革又要再加一层复杂度。

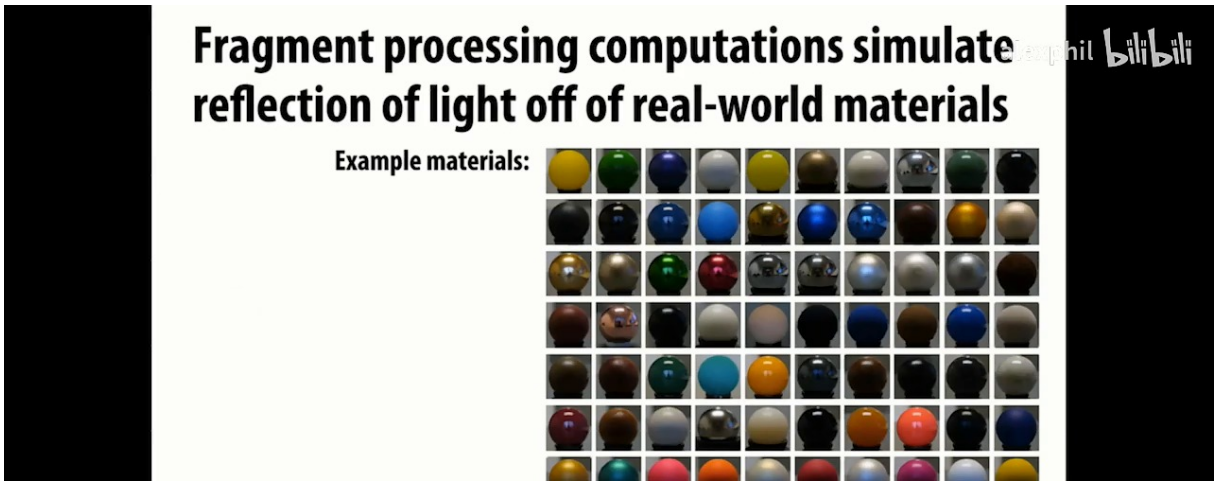


图 7: 片元处理：模拟光线在真实材质上的反射[7]



图 8: 现实世界材质与光源的极大多样性[8]

也有取巧的办法。桌布有规则的图案，于是可以用纹理映射：把一张平铺图案投影到物体表面。这以极低成本伪造出复杂图案的观感。

[7] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

2.4 着色器：可编程的管线阶段

讲者由此得出关键结论：不应把整条管线都固化在硬件里。每次增加新特性，你可能都想用软件来描述要执行什么处理。

于是出现了着色器。它是在片元生成之后运行的一小段代码，例如给远处像素调整颜色，模拟光线穿过的扩散效果。讲者强调这是一种相对局部的处理——不必真的模拟光的物理。

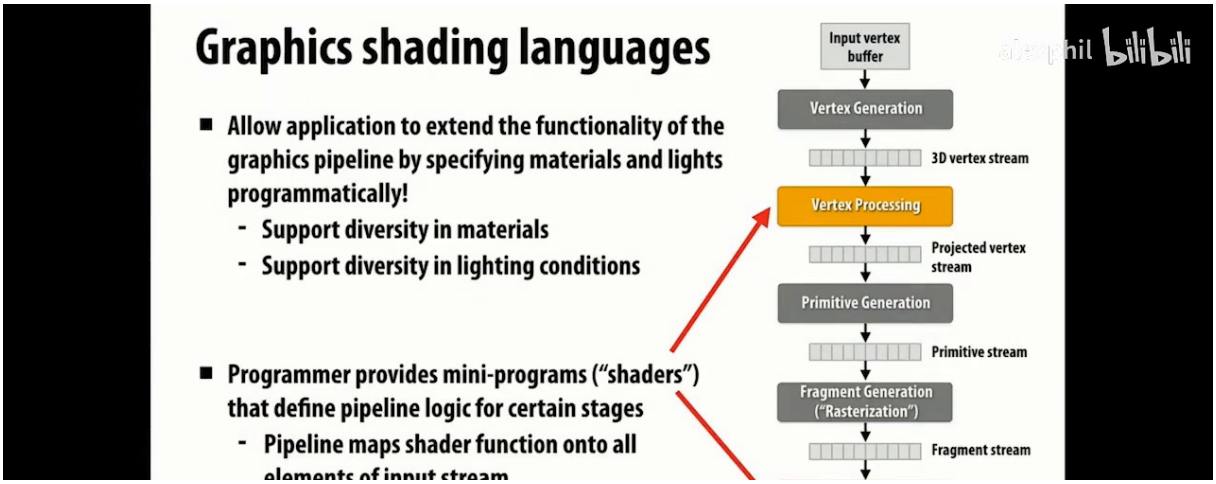


图 9: 图形着色语言：顶点与片元着色器可替换管线阶段[9]

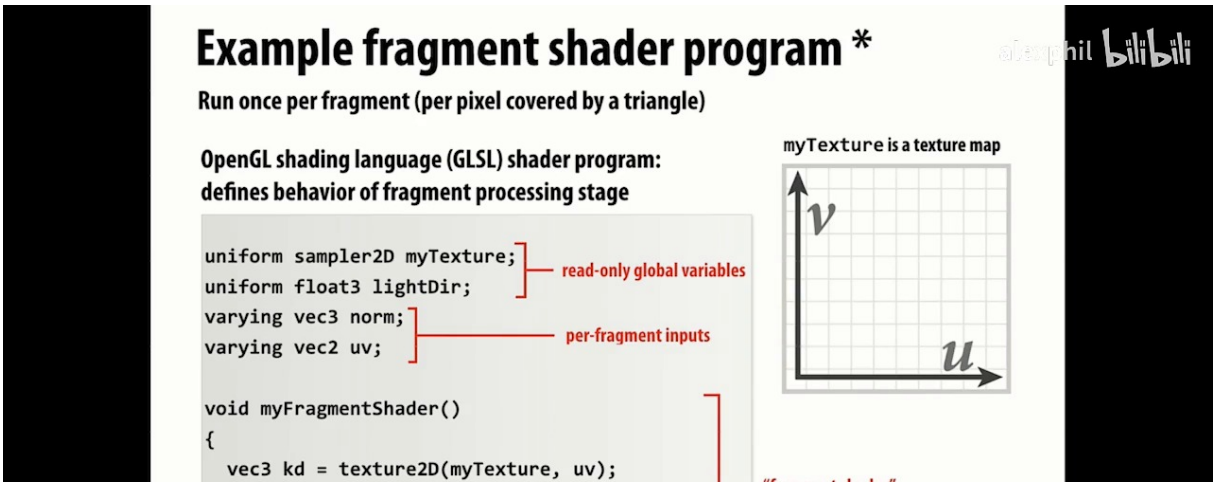


图 10: 片元着色器示例：每个片元运行一次[10]

2.5 本章小结

图形管线把渲染拆成顶点、三角形、片元、像素几个阶段，每个阶段都是对一堆独立对象做同样的操作。材质的复杂度推动管线不断加长，而着色器让这些阶段可以用软件替换。这个”对大量独立数据执行同一段程序”的结构，正是后来 CUDA 抽象的原型。

[9] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。
[10] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。

3 从图形卡到通用计算

3.1 历史背景

讲者回顾：这类工作早年都由普通 CPU 完成，2004 年单核无法继续提速，但晶体管数量仍能增加。

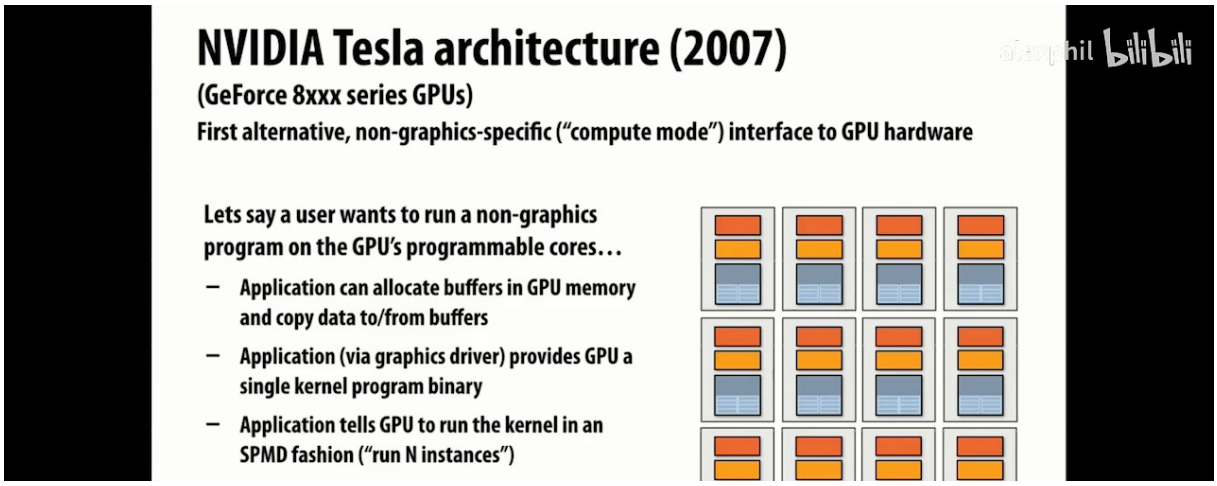


图 11: NVIDIA Tesla 架构（2007）：第一个非图形的替代方案[11]

GPU 由此诞生：可以造出大量处理单元，只是不能让单个巨型处理器跑单一指令流继续变快。

早期 GPU 纯粹为实现图形管线而设计。它们可编程，但只限于与图形操作直接相关的约束范围内——是可编程的专用硬件。

3.2 把科学计算伪装成图形操作

讲者讲了一段有意思的历史。一些聪明人意识到这东西能力很强，只要肯" 倒立着思考"，就能把许多经典计算问题投射成图形操作。

抽象的形式由硬件能力决定

GPGPU 时代把计算伪装成图形操作，不是因为这些映射自然，而是因为当时的硬件只暴露图形接口。这说明抽象的形状往往由可用硬件的形态倒推出来，而不是从问题本身推出。识别这一点，才能判断某个抽象何时应当被替换。

第一个障碍是它只能处理三角形。解决办法很直接：任何方块都能用两个三角形覆盖。其次，可以创建特定尺寸的数组，并把其中的值当作数字使用。

于是有大量工作证明这些专用设备能解决科学问题、做科学模拟。

[11] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

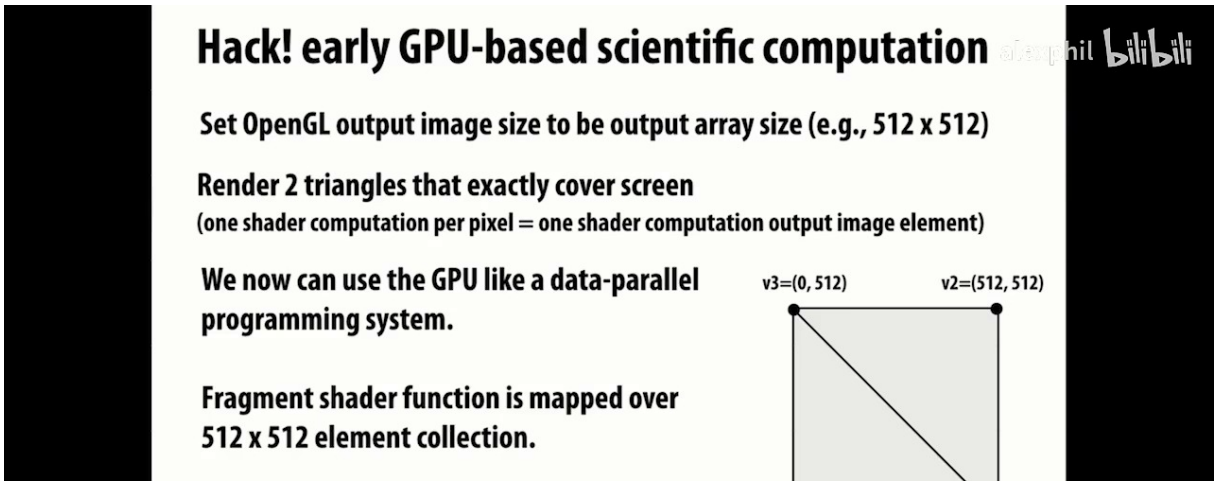


图 12: 早期 GPU 科学计算的技巧：把输出图像当作数据、把计算伪装成图形操作[12]

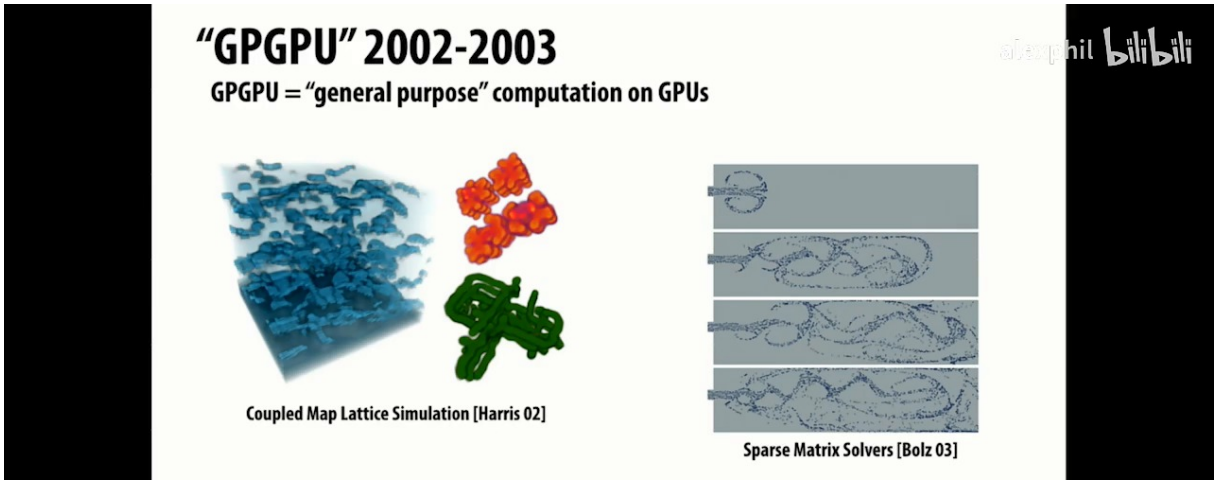


图 13: GPGPU 2002 到 2003：通用计算在 GPU 上的萌芽[13]

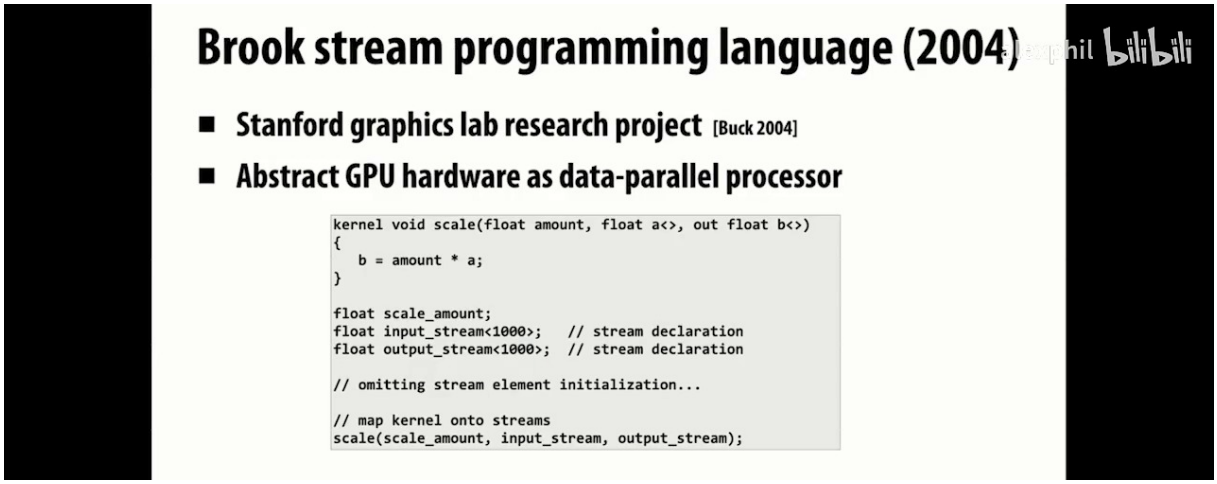


图 14: Brook 流编程语言（2004）：斯坦福图形实验室的研究项目[14]

[12]视频画面时间区间：00:22:00–00:22:15。若为裁剪图，时间区间仍对应原视频画面。

[13]视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

这一段的历史意义在于：它证明了 GPU 的硬件形态足以承担通用计算，问题只在于编程方式太别扭。

3.3 为什么不能让操作系统调度

讲者在这里做了关键对比。CPU 上跑代码，是操作系统控制调度：它把程序加载进内存、选择哪个核执行、用中断机制不断切换进程，以支持多任务、IO 与虚拟内存这些可能造成长时间暂停的操作。

这套机制很重。讲者给出的数字是：一次 pthread 的上下文切换要花 10000 到 20000 个时钟周期——虽然绝对时间只有微秒级，但对一个追求高效利用硬件的程序来说，这个比例完全不可接受。

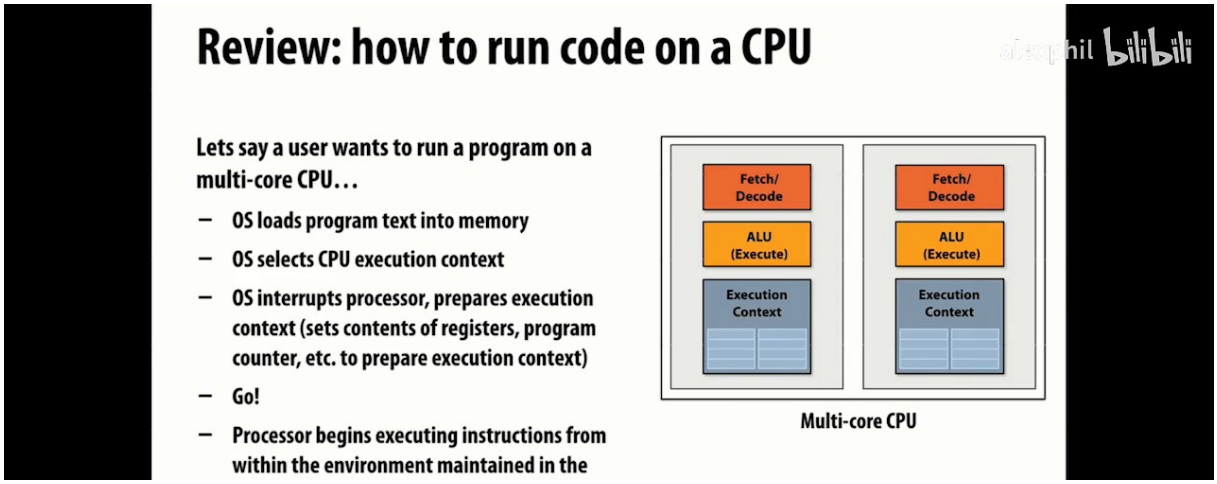


图 15: 回顾：代码如何在 CPU 上运行，操作系统掌控调度[15]

早期在 GPU 上运行的方式则是：把配置信息加载进 GPU，告诉它每个阶段该做什么，然后说”开始”，管线就跑起来。但这不被视为频繁操作——配置好就让它一直跑。

新的方向要让 GPU 支持非图形应用，但必须绕开操作系统：硬件的原始能力很强，可调度决策必须由硬件自己掌控。

[14] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。
[15] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

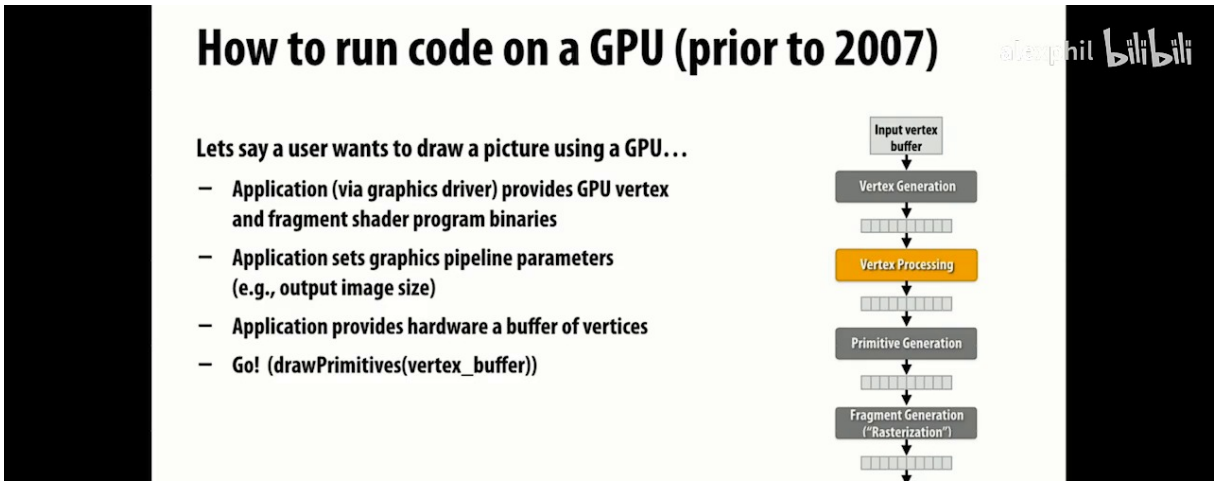


图 16: 2007 年以前代码如何在 GPU 上运行：配置管线后一次性启动[16]

3.4 CUDA 的定位

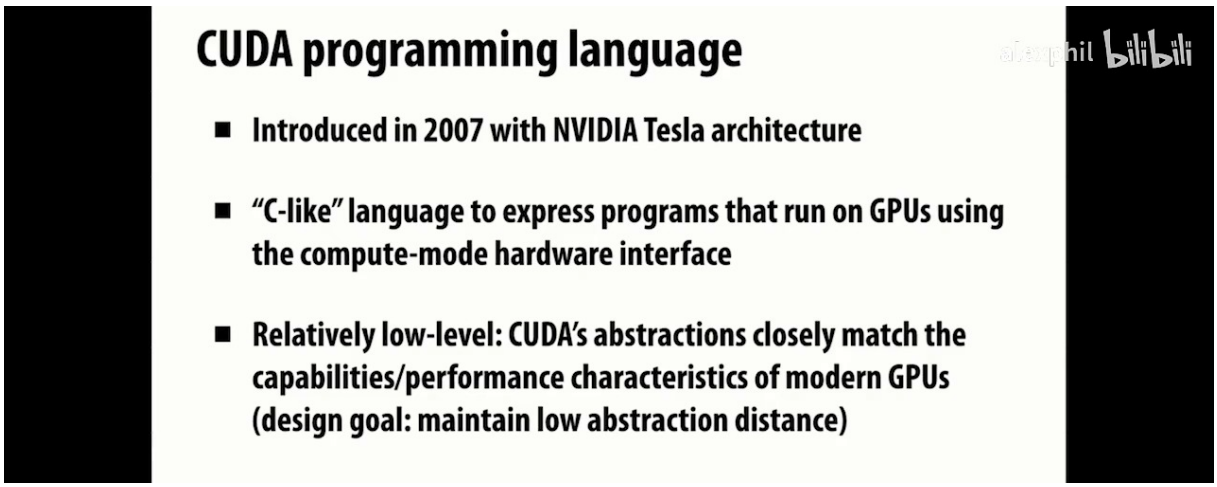


图 17: CUDA 编程语言：2007 年随 Tesla 架构推出[17]

CUDA 由 NVIDIA 在约十年前推出。写起来你会隐约觉得在写 C 或 C++，但并不是——你在描述一种高度数据并行的计算。

讲者认为 CUDA 做得好的一点是：既给出足够高层的抽象模型作为起点，又提供足够的接口让你能下探到 GPU 内部去调优。而这种调优是可以增量进行的——先看时间花在哪里，再决定改什么。他还建议始终保留一个功能正确的"黄金实现"，即使慢，也能在每次调优后对照验证结果。

他也坦率指出缺点：CUDA 随硬件架构演进不断变化，年度之间的兼容性不佳。开源的对应物 OpenCL 外观很像，但 CMU 的课程通常偏好开源方案以避免绑定单一公司——这一处是例外，专有版本确实明显更好、使用也更广。

[16] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:30:00–00:30:15。若为裁剪图，时间区间仍对应原视频画面。

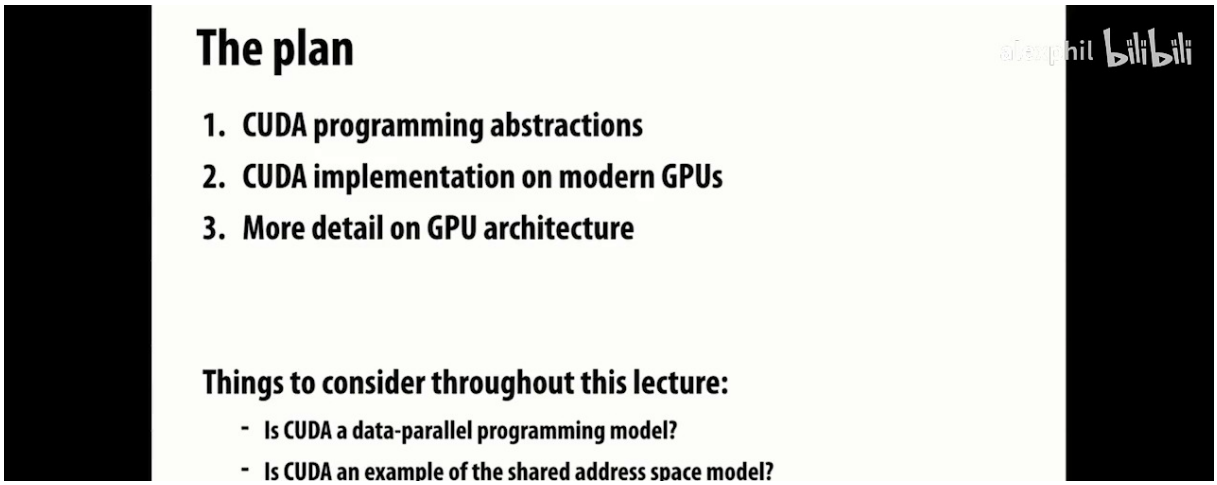


图 18: 本讲计划：CUDA 抽象、硬件映射与性能细节[18]

3.5 代价：功耗与价格

有学生问 GPU 为什么没撞上功耗墙。讲者回答它们也撞了——GHC 集群加上这些 GPU 后不得不增加空调能力。GPU 功耗约 180 瓦，芯片也很大，高端型号价格量级相当于数十万美元，500000 美元一台的高端卡并不罕见（视频 00:42:39）。

真正的优势在于可以扔掉多年来为榨取顺序程序并行度而添加的大量控制逻辑：庞大的分支预测、乱序执行、推测执行，以及大量缓存。扔掉这些，芯片面积就能腾出来做加法与乘法。

GPU 不是绕过功耗墙，而是换了一条花钱方式

GPU 的功耗与价格都远高于 CPU，它并没有解决散热问题，只是把每瓦预算从控制逻辑转移到了算术单元。因此在评估是否使用 GPU 时，不能只看峰值算力，还要看功耗预算、散热能力与采购成本是否可接受。

3.6 本章小结

GPU 从固定管线走向可编程，再被证明能承担通用计算。关键的技术障碍是调度：操作系统的上下文切换开销对 GPU 的利用方式无法承受，因此调度必须由硬件完成。CUDA 由此出现，提供高层抽象又能下探调优。它的优势来自砍掉控制逻辑，代价是功耗与价格。

4 CUDA 的抽象：线程、块与网格

4.1 先澄清一个术语陷阱

讲者提醒：同一个术语在不同语境下含义差别很大，其中”线程”尤其如此。

pthread 的线程是非常独立的执行流：有自己的控制、可以有自己的数据、与其他线程几乎无耦合。

CUDA 线程完全不同。它是程序在一份数据上的执行。多个 CUDA 线程彼此强耦合，能做的交互形式也受限得多。讲者说接下来的内容里，”线程”一律指 CUDA 线程。

[18] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

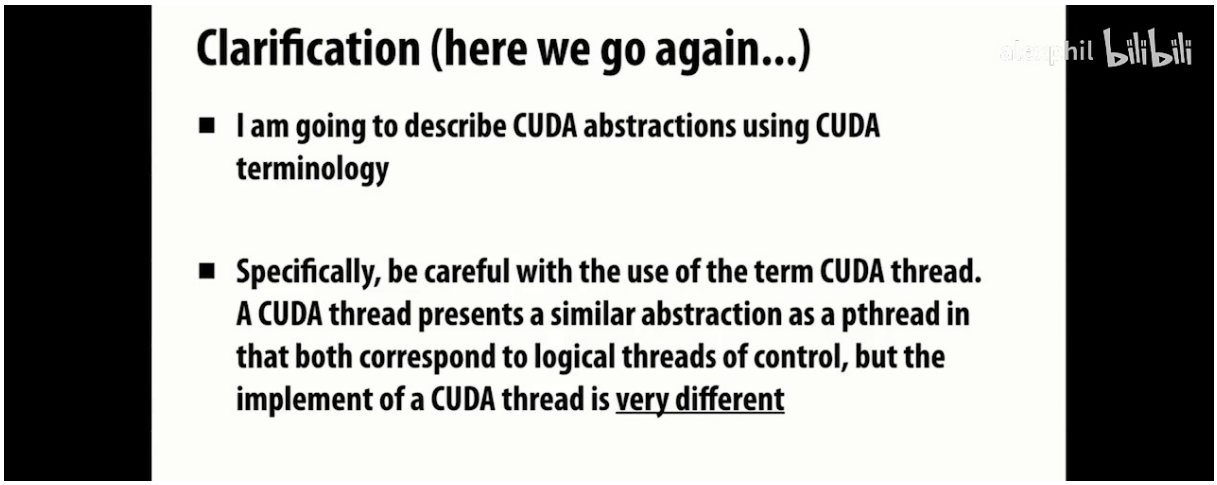


图 19: 术语澄清：CUDA thread 与 pthread 语义差别极大[19]

4.2 三维的线程索引

在高层看，CUDA 是数据并行模型：一个线程就是对某个数据元素的处理。CUDA 支持最多三维的矩阵或数组表示，并为此提供直接支持——不过讲者强调，这更多是一种错觉，硬件里并没有三维结构。

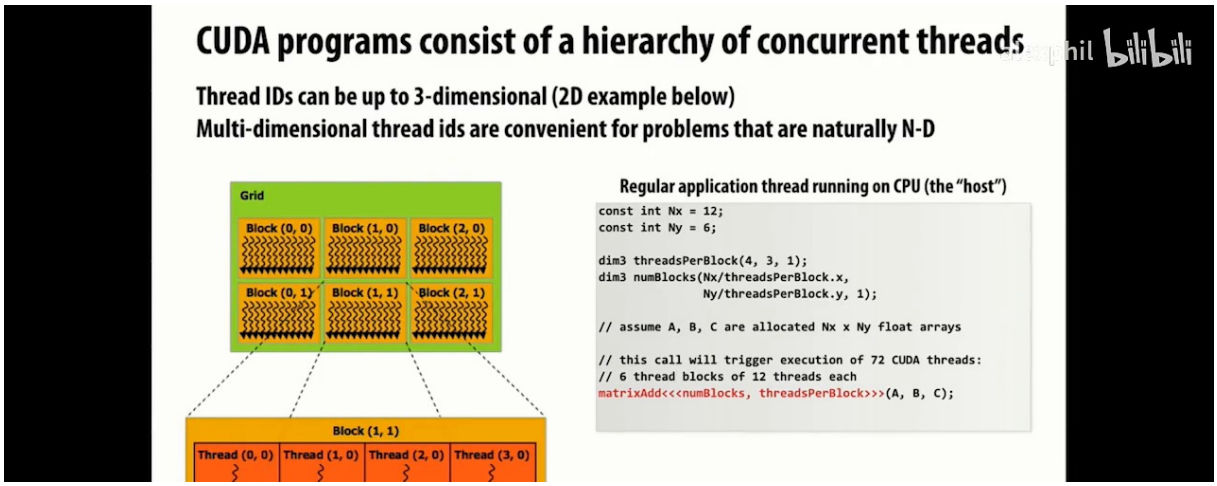


图 20: CUDA 程序由并发线程的层次结构组成，线程索引最多三维[20]

4.3 block 与 grid

以矩阵加法为例：一个 12×6 的矩阵，划分成若干 block，每个 block 是四列宽、三行高。于是得到六个块。每个 block 内是若干线程，每个线程负责一个元素。

若一个 block 是 4×3，那么所需的 block 数就是行数除以每块行数、列数除以每块列数。

写成式子：

[19] 视频画面时间区间：00:33:00–00:33:15。若为裁剪图，时间区间仍对应原视频画面。

[20] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

$$N_{\text{block}} = \left\lceil \frac{N_x}{B_x} \right\rceil \times \left\lceil \frac{N_y}{B_y} \right\rceil$$

- N_x, N_y：数据在 x 与 y 方向的元素数，本例为 12 与 6；
- B_x, B_y：每个 block 在两方向上的线程数，本例为 4 与 3；
- 向上取整保证数据不被 block 尺寸整除时仍能覆盖全部元素。

讲者指出这背后是 2-13 里学过的行主序多维数组概念。

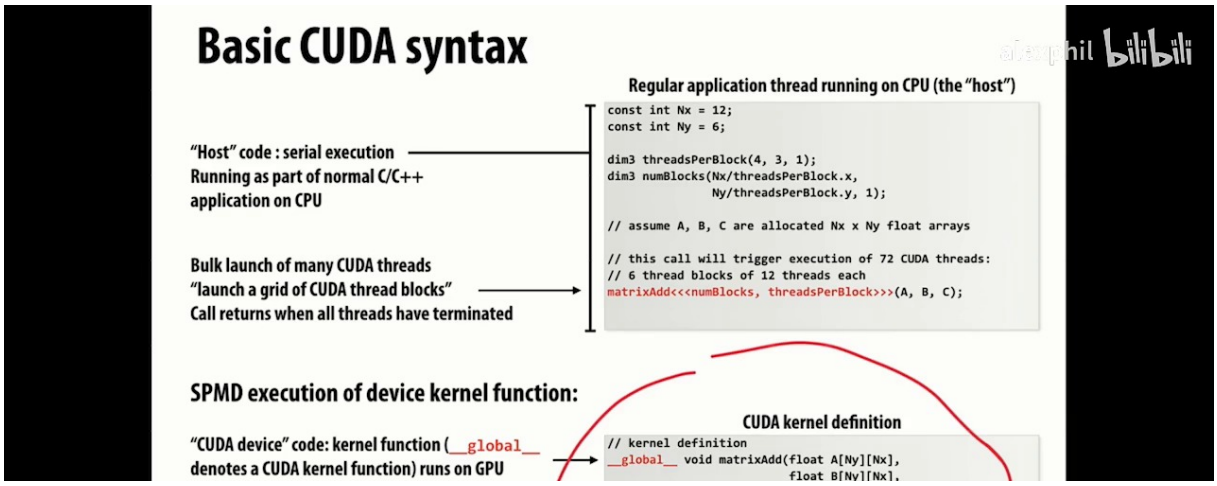


图 21: CUDA 基本语法：host 端配置 block 结构并启动核函数[21]

4.4 核函数

GPU 执行的那段代码叫核函数。它的写法基于图形学的思路：把自己想象成矩阵的第 (i, j) 个元素，做 a_{ij} 加 b_{ij} 并存到 c。

核函数需要一个全局下标，由三个量算出来：

$$i = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}$$

- `blockIdx`：本 block 在 grid 中的编号，由 CUDA 在启动时分配；
- `blockDim`：每个 block 的尺寸，本例中 x 方向为 4；
- `threadIdx`：本线程在 block 内的编号；
- i：对应的全局元素下标。

讲者强调，核函数启动时它只知道自己是这个 block 里的某个线程，而这个 block 是众多 block 之一。

[21] 视频画面时间区间：00:38:00–00:38:15。若为裁剪图，时间区间仍对应原视频画面。

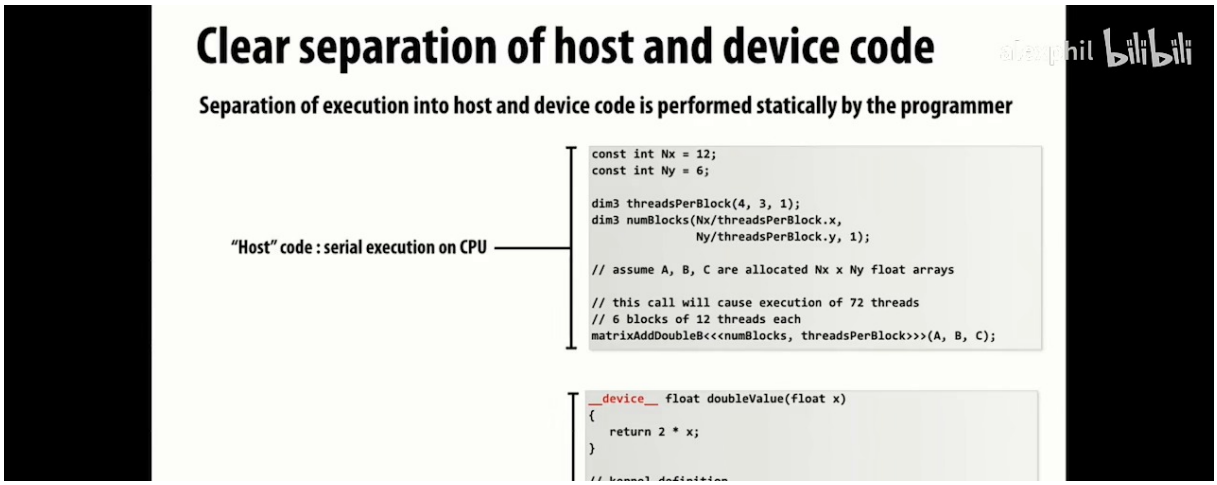


图 22: host 与 device 代码的清晰分离[22]

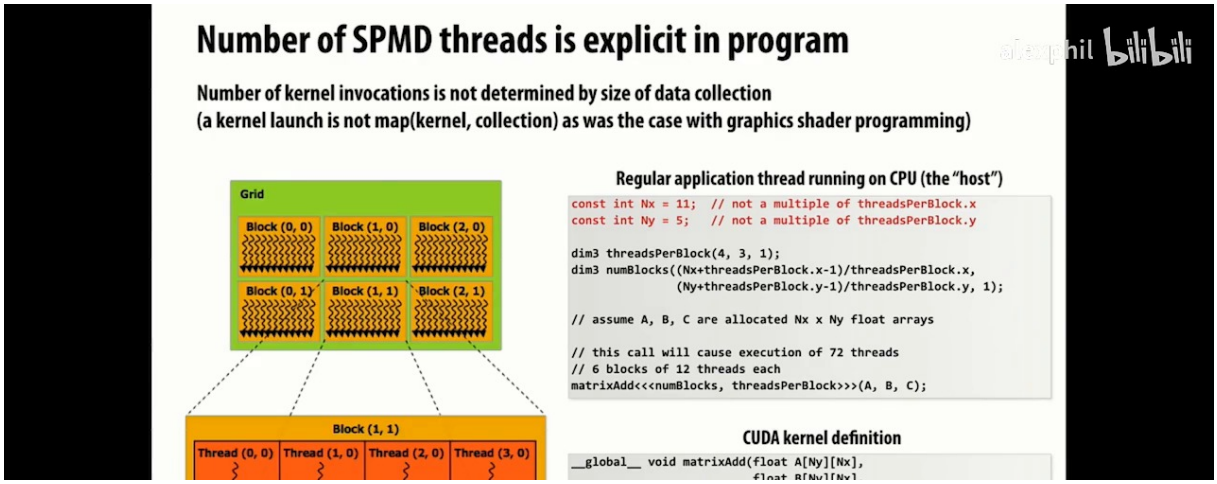


图 23: SPMD 线程数在程序中显式给出[23]

4.5 三个修饰符

同一份文件里既有 host 代码也有 device 代码，靠修饰符区分。

`__global__` 标记核函数，从 host 端调用、在设备上执行。`__device__` 标记设备端调用的普通函数，看起来就像普通函数。host 端则可以执行任意 C/C++，但会做一种特殊操作——线程启动，即那对三尖括号 `<<<...>>>`，用来指定 grid 与 block 的尺寸。

三尖括号是 host 与 device 的分界线

`kernel<<<gridDim, blockDim>>>(...)` 这一行是唯一同时属于两个世界的地方：由 host 发出，却描述设备上的线程拓扑。它不返回执行结果，只发出启动请求；设备上的执行与 host 的后续代码可以并行进行，需要显式同步才能保证顺序。

[22] 视频画面时间区间：00:44:00–00:44:15。若为裁剪图，时间区间仍对应原视频画面。

[23] 视频画面时间区间：00:45:00–00:45:15。若为裁剪图，时间区间仍对应原视频画面。

4.6 block 尺寸的硬限制

block 内线程数有硬件上限

讲者指出单个 block 的总线程数上限是 1024，这是硬件限制。实际写代码时还应优先选 32 的倍数这类对硬件友好的尺寸，因为线程会被按固定宽度分组执行。超出上限或尺寸不佳时，要么编译失败，要么执行效率下降。

学生问 block 是不是纯粹的虚拟概念。讲者回答：block 在组织计算的意义上是虚拟的，但它有上限，而且每个 block 会被映射到一个执行单元（SM）上执行。所以它更像是对计算的切块，而不是某块具体硬件；CUDA 让你看到硬件的这一点点结构，是为了方便后续调优。

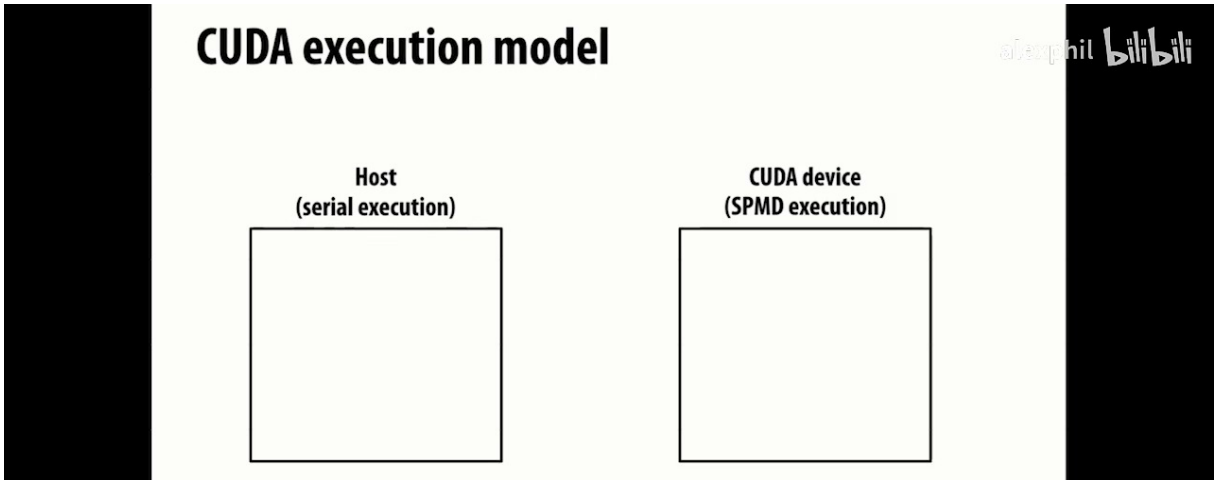


图 24: CUDA 执行模型：host 串行执行，device 以 SPMD 方式执行[24]

4.7 越界处理

如果数据规模不是 block 尺寸的整数倍，代码必须处理越界。讲者说这是很标准的做法：判断下标是否在范围内，越界就不做事。

他的分析值得记住：这个检查几乎零成本。唯一的代价是没有用满所有功能单元。这与 CPU 上的情形根本不同——在 CPU 上每个条件判断都可能是一次分支预测失败，是巨大减速的来源。而在这里，我们本来就有远多于硬件的活儿要干，因此调度可以相当粗放，仍能最大化利用性能。

越界检查在 GPU 上便宜的原因

GPU 的线程远多于硬件通道，多余线程被禁用只是让某些批次空转，不产生分支预测失败的惩罚。而在 CPU 上条件分支会打断流水线，代价高得多。同一个习惯用法在两种硬件上的成本完全不同，这是不能把 CPU 优化直觉直接搬到 GPU 的原因之一。

4.8 本章小结

CUDA 用 grid、block、thread 三级结构描述并发，核函数内用 `blockIdx` 与 `blockDim` 与 `threadIdx` 算出全局下标。block 上限 1024 线程，每个 block 映射到一个 SM。CUDA 是 SPMD 而非 SIMD，硬件负责把海量线程映射到有限通道上，因此程序员可以只写高层代码。

[24] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

抽象既然回避了硬件，性能就被藏了起来。要把它拿回来，必须理解内存层次——那是下一章的主题。

5 内存层次与共享内存

5.1 地址空间是分离的

CUDA 内存模型最需要记住的一点：host 与 device 有各自独立的地址空间。

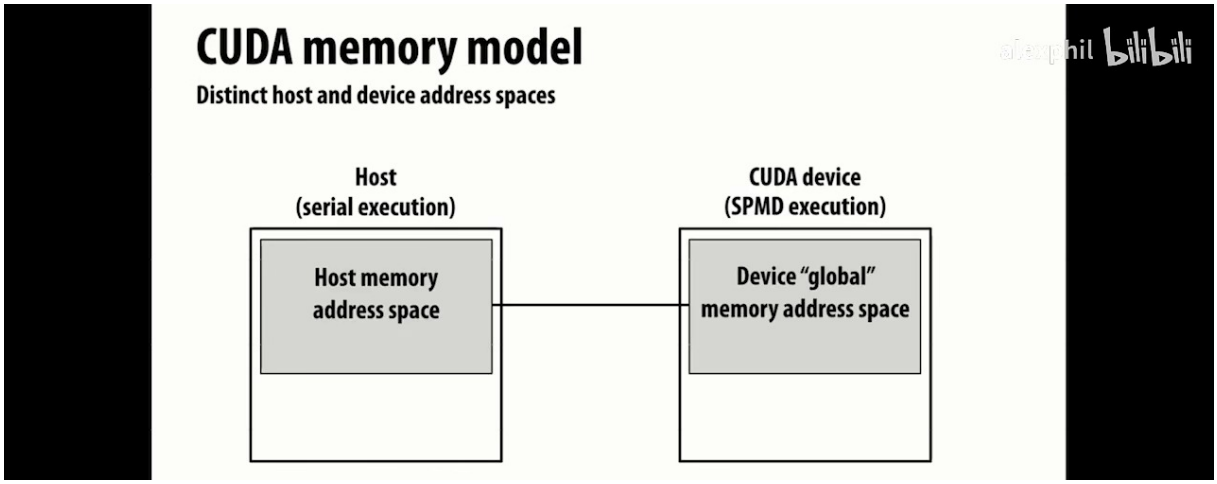


图 25: CUDA 内存模型：host 与 device 地址空间彼此独立[25]

跨地址空间的拷贝是最容易漏掉的成本

每次把数据搬到设备、把结果搬回主机，都要占用 PCIe 带宽，且需要显式同步。若内核本身的计算量很小，拷贝时间可能超过计算时间，使得 GPU 版本比 CPU 还慢。优化 GPU 程序时，先算清计算与传输各占多少时间，再决定是否值得搬到设备上。

因此数据必须在代码里显式搬运。讲者介绍了 `memcpy` 原语，用于在两个地址空间之间移动数据。

[25] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

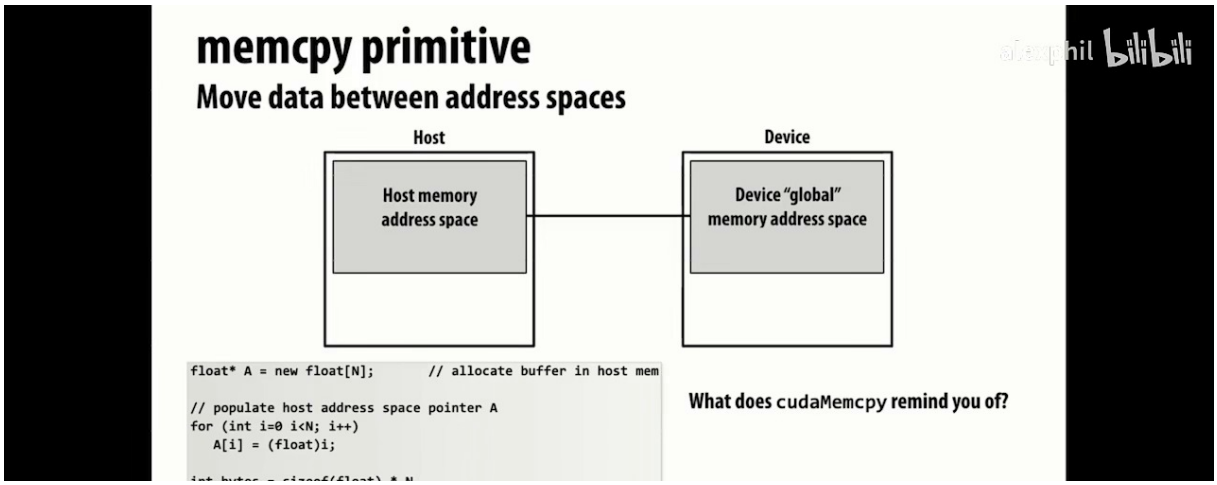


图 26: memcpy 原语：在 host 与 device 地址空间之间搬数据[26]

分配设备内存用 `cudaMalloc`。讲者特意指出它与常规 malloc 的差别：`cudaMalloc` 不返回指针，而是返回错误码，真正的指针通过参数传出。他顺带评论了早期 C 库以返回值表示结果的做法，以及为什么现代库改用错误码。

设备内存容量在 4 GB 到 8 GB 这个量级（视频 00:51:10）——不算大，但也不差。讲者也强调，这里的并发模型与 pthread 大不相同，对内存操作的一致性假设必须非常小心。

5.2 三类内存

讲者给出内核可见的三类内存。

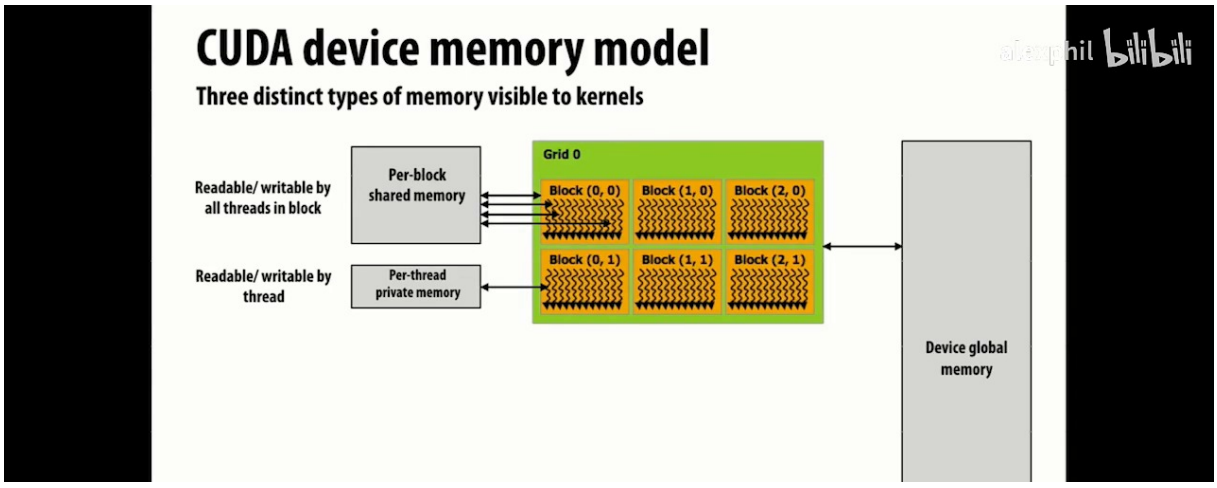


图 27: CUDA 设备内存模型：内核可见的三类内存[27]

第一类是全局内存，所有线程都能访问，容量最大、延迟最高。第二类是每个 block 的共享内存，由该 block 的所有线程共同使用。第三类是每个线程的极少私有存储。

关键差异在于控制权。CPU 的缓存完全由硬件控制，程序员无法指定什么留在缓存里。GPU 上这些内层存储更受软件控制，可以当作" 软件缓存" 使用：由你的代码决定往里放什么。

[26] 视频画面时间区间：00:52:00–00:52:15。若为裁剪图，时间区间仍对应原视频画面。

[27] 视频画面时间区间：00:53:30–00:53:45。若为裁剪图，时间区间仍对应原视频画面。

共享内存不会自动装数据

读一次全局内存，数据不会自动出现在 block 的共享内存里。必须由代码显式写入。这是它与硬件缓存的根本区别：缓存自动生效，共享内存需要你主动管理。正因如此，它会带来额外的编程负担，但也带来了缓存无法提供的确定性。

5.3 本章小结

CUDA 的 host 与 device 内存彼此独立，必须显式拷贝；`cudaMalloc` 返回错误码。设备端有三类内存：全局、block 共享、线程私有，其中共享内存由软件管理，是可控的” 软件缓存”。

这套内存模型解释了一件事：GPU 上重复的全局读取没有缓存来救。下一章用卷积的例子把这个代价算清楚。

6 卷积示例：从 3n 次读取到一次

6.1 版本一：一个线程一个输出

讲者引入一维卷积作为贯穿示例。版本一的策略是一个线程负责一个输出元素，block 尺寸取 128。

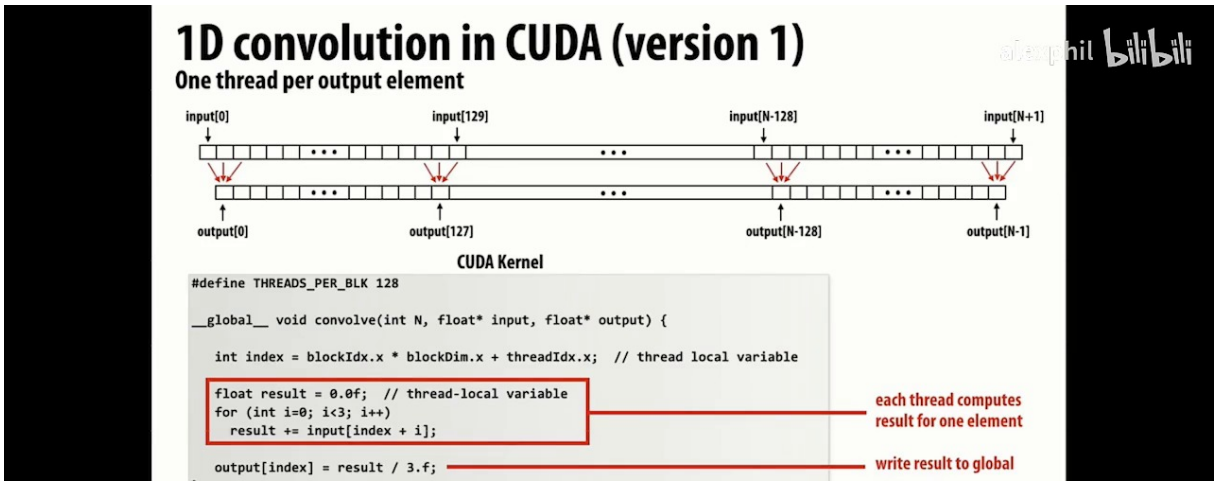


图 28: 一维卷积版本一：每个输出元素一个线程[28]

这段代码能正常工作，但不够快。讲者引导学生算一笔账，并特意叮嘱：不要假设有缓存。

答案是 $3n$ 次全局读取——每个线程都要做三次读取，而且它们之间不共享。因为不存在缓存，每次读就是真的去读，慢就是慢。

于是出现了一个明显的低效：三个线程都需要同一个输入值，它们却完全没有协作，各自独立地读了一遍。

[28] 视频画面时间区间：00:56:30–00:56:45。若为裁剪图，时间区间仍对应原视频画面。

重复的全局读取是 GPU 内核最常见的浪费

在 CPU 上，同一地址的多次读取可能被缓存吸收，因此这类浪费常被掩盖。GPU 的全局内存没有这种自动机制，重复读取全部落到带宽上。判断一个内核是否低效，先数它对全局内存做了多少次访问，再问这些访问能否被合并。

6.2 版本二：先用共享内存装一遍

既然 block 内有共享内存，就可以把它当缓存用——但必须显式使用。

讲者展示了版本二：在核函数里声明一个局部数组，用 `__shared__` 修饰。这样这块内存被分配在处理器（SM）的共享内存里，而不是全局内存。

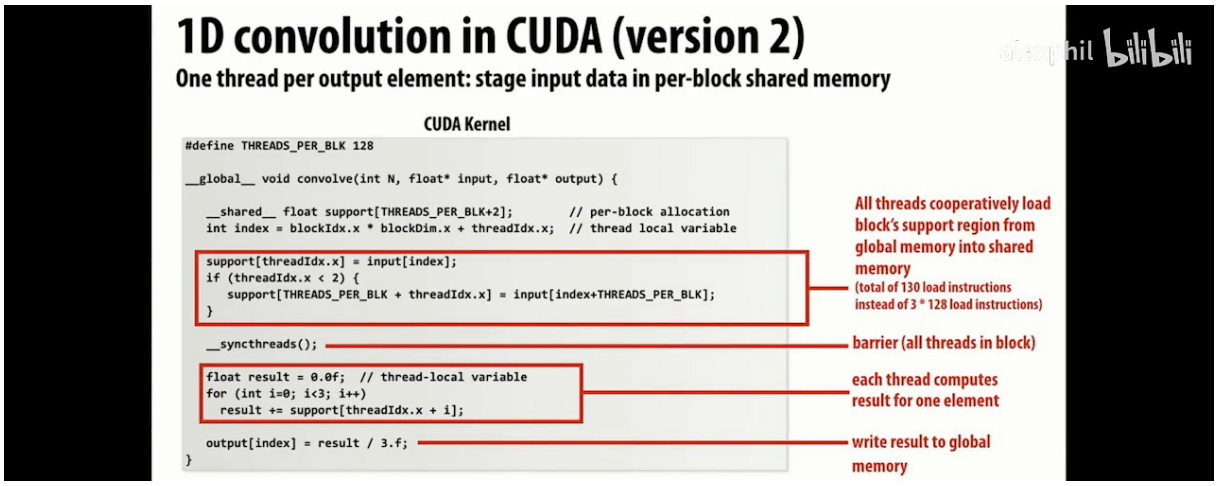


图 29: 一维卷积版本二：先用共享内存暂存输入数据[29]

代码分两个阶段。第一阶段每个线程从全局数组读入自己需要的那一段，写进共享数组；第二阶段所有线程改用共享数组做卷积计算。

由于一个 block 的所有线程共用这块共享内存，同一个输入值现在只被从全局读了一次，而不是三次。

把两种版本的全局读取次数写成式子是：

$$R_{\text{naive}} = 3n, \qquad R_{\text{shared}} \approx n$$

- R_{naive}：版本一的全局读取次数，每个输出做三次读；
- R_{shared}：版本二装载阶段总共读入的元素数，约为 n 加上块间重叠的少量边界；
- n：输入元素总数。

节省的比例接近 3 比 1，而这正是 GPU 上最稀缺的资源——全局内存带宽。

[29] 视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。

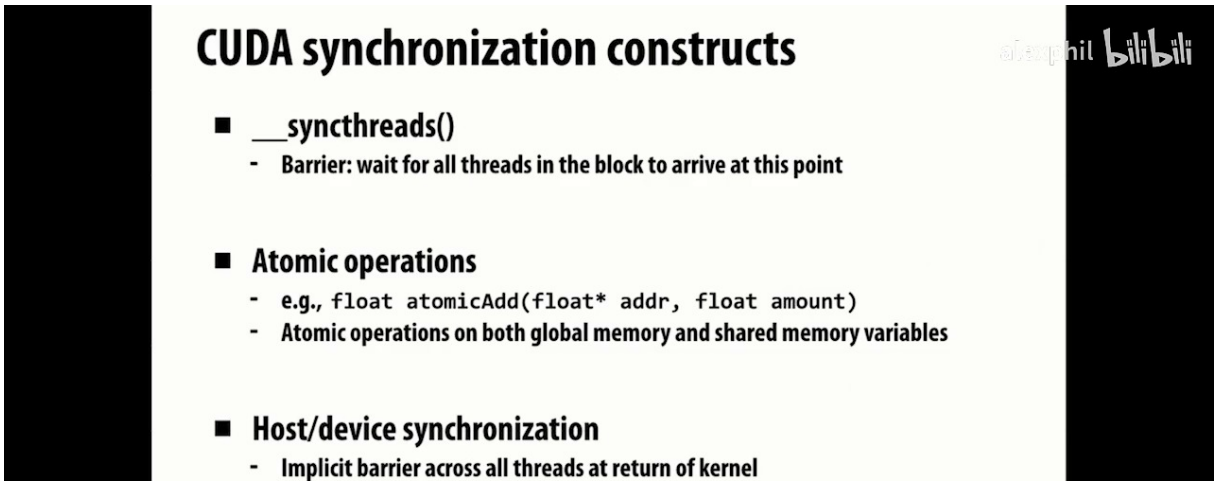


图 30: CUDA 同步原语 ___syncthreads：等待 block 内所有线程到达屏障[30]

6.3 为什么必须同步

两个阶段之间需要 `__syncthreads()`。它是一条屏障：等待 block 内所有线程都到达此处再继续。

理由与此前的红黑格序一致——如果不等待，第二阶段可能有线程读到了别人尚未写入的共享内存，结果是错的。讲者用”相当巧妙”形容这段代码，正是指这种手工分阶段加屏障的写法。

共享内存优化必须配同步

`__syncthreads()` 只保证 block 内同步，不跨 block。若误以为它能同步整个 grid，跨 block 的数据依赖就会出错而没有任何报错。此外屏障必须被 block 内所有线程执行到，放在条件分支内会导致死锁。

6.4 本章小结

朴素实现的每个输入需要 $3n$ 次全局读取，因为同一数据被多个线程各自读了一遍。用 `__shared__` 把输入按 block 装一次、再用 `__syncthreads()` 分阶段，可以把重复读取降到一次。这一步手工管理正是换取性能的代价。

讲者随后总结：CUDA 的高吞吐模型让”提高吞吐比降低延迟容易”这句话落到了实处。

[30] 视频画面时间区间：01:07:30–01:07:45。若为裁剪图，时间区间仍对应原视频画面。

7 抽象回顾与执行细节

7.1 CUDA 抽象的两层

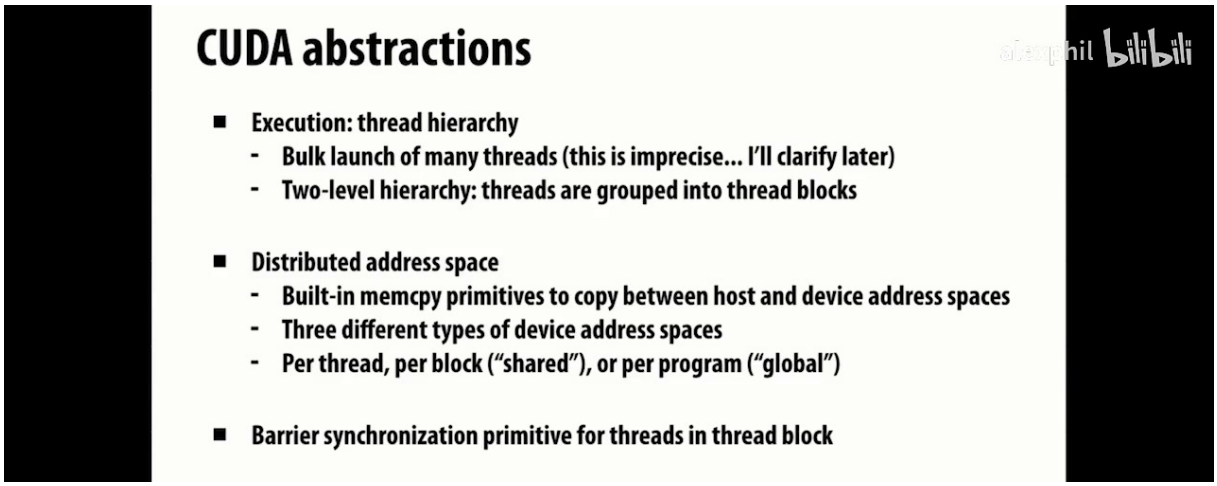


图 31: CUDA 抽象汇总：执行层次与内存层次[31]

CUDA 的抽象可以分成两条轴。执行轴上是从 thread 到 block 到 grid 的层次，核心特征是批量启动大量线程。内存轴上是从线程私有到 block 共享到全局的分层，核心特征是后两者可由软件控制。

层次	作用域	谁管理	容量与速度
线程私有	单个 CUDA 线程	编译器	极小、最快
block 共享	一个 block 内全部线程	程序员显式	小、快
全局内存	整个 grid	程序员显式	大、慢

表 1: CUDA 三类内存的作用域与管理方式

7.2 warp 与 SMM

讲者最后展示了执行细节。以 2014 年的 GTX 980（Maxwell 架构）为例，其中每个 SMM 单元（讲者的引号表明这相当于一个” 核”）都有若干 warp。

[31] 视频画面时间区间：01:08:30–01:08:45。若为裁剪图，时间区间仍对应原视频画面。

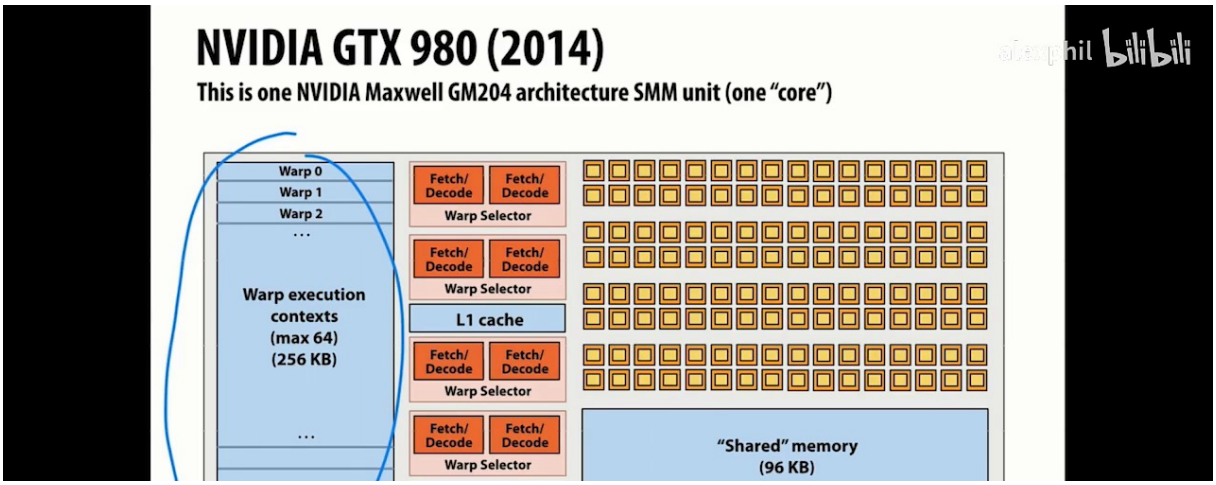


图 32: NVIDIA GTX 980（2014）的 Maxwell SMM 单元[32]

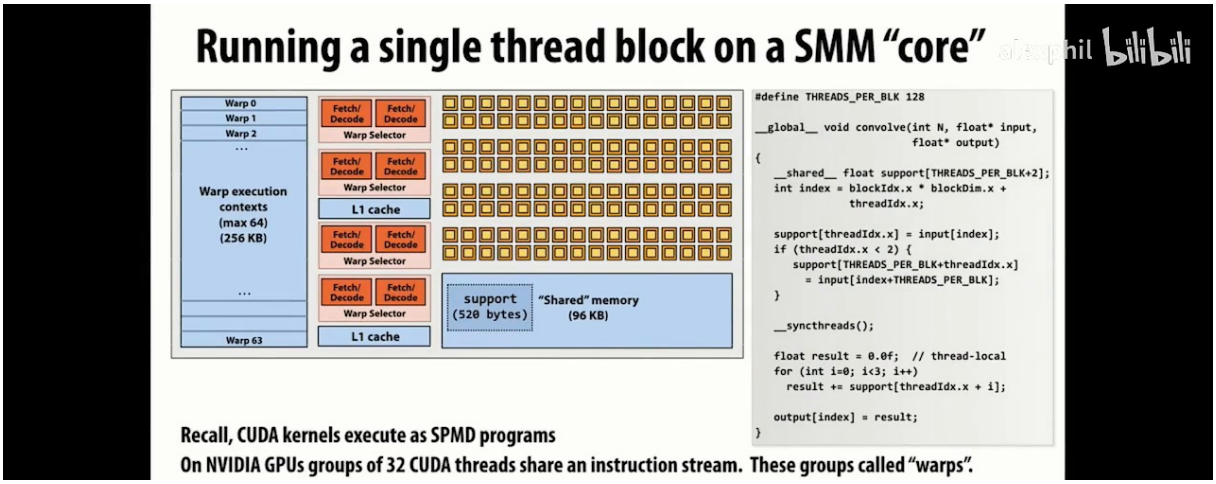


图 33: 在一个 SMM 核上运行单个线程块：线程被组织成若干 warp[33]

讲者展示了一个含 128 线程的 block 如何在 SMM 上运行。这 128 个线程被组织成固定宽度的 warp，warp 是硬件调度的基本单位。这也解释了前面为什么建议 block 尺寸取 32 的倍数：若尺寸不是 warp 宽度的整数倍，最后一个 warp 会有空闲通道。

warp 是 CUDA 隐藏的关键执行单位

线程在抽象上是独立的，但硬件按 warp 成组执行。同一 warp 内的线程若走不同分支，就会依次执行各分支、互相等待，这就是 warp 内的分歧。因此 CUDA 代码中把相邻线程安排去处理相邻数据，可以让同一 warp 的行为保持一致，避免分歧。

7.3 本章小结

CUDA 的抽象沿执行与内存两条轴展开，block 是这两条轴的交点：它既是调度单元，也是共享内存的作用域。硬件层面线程按 warp 组织，这决定了 block 尺寸与数据排布的选择。

到这里，写出一份正确且高效的 CUDA 内核所需的全部要素都已出现。

[32] 视频画面时间区间：01:14:00–01:14:15。若为裁剪图，时间区间仍对应原视频画面。

[33] 视频画面时间区间：01:16:00–01:16:15。若为裁剪图，时间区间仍对应原视频画面。

8 总结与延伸

讲者的收尾带着务实的态度：即使只有 10% 的效率，你跑得也很快；真正的性能调优者能把利用率推上去，但你往往会发现推不上去。不过只要计算能被表达成对独立数据做同样的操作，你就能让它在 GPU 上跑得很好。他还承诺在作业中会遇到一些看似不独立、但可以通过原语操作改写的情况。

把全讲串起来：GPU 为实时三维渲染而生，图形管线天然是数据并行，因此硬件形态与这条管线同构。真实感需求推动管线加长，着色器让阶段可以用软件替换，最终引来把科学计算伪装成图形操作的 GPGPU 时代。操作系统的调度开销（一次 pthread 上下文切换上万时钟周期）使 GPU 必须自己做调度，CUDA 由此出现。它用 grid、block、thread 描述并发，用 host 与 device 分离的地址空间和显式拷贝处理数据流，用三类内存给出可控的局部性手段。卷积示例把这一切落到具体数字上：朴素版本需要 $3n$ 次全局读取，用 `__shared__` 加同步可以降到一次。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- CUDA 的抽象刻意隐藏了硬件，性能必须靠下探找回。讲者展示的 block、共享内存、warp 都不是纯粹的抽象概念，而是让程序员能够干预硬件的接口。用高层抽象写出的代码正确但慢，这不是缺陷而是设计意图。
- 共享内存是可控性的来源，也是同步负担的来源。缓存自动生效，共享内存需要手动装载与屏障。因此它并非总是更优：当数据访问本来就有良好局部性时，硬件缓存可能足够，手工管理反而增加出错机会。
- warp 的存在解释了多个看似无关的建议。block 尺寸取 32 的倍数、相邻线程处理相邻数据、避免 warp 内部分歧，这些都源于硬件按固定宽度成组执行。
- 10% 效率仍可能很快，这句话要放在绝对速度上理解。与第七讲（算术强度）的结论一致：峰值利用率低不代表不划算，比较要看绝对耗时。

这堂课的边界在于它是 GPU 入门，讲者明确表示调优细节与更多原语会在作业里遇到。因此本文给出的是 CUDA 的概念地图与第一个可量化的优化范式，不含完整的最佳实践清单。

两个可以直接从本讲数据提出的下一步问题：其一，共享内存容量有限，一个 block 能装载的数据量有上限，那么当卷积核更长时该如何分块？其二，warp 内的分歧会让同一批线程串行执行各分支，那么怎样的数据排布能让同一个 warp 尽量走同一分支？这两个问题分别指向下一讲的局部性优化与 GPU 性能调优。